Interfaces and Inheritance

A Game of Chess

```
public class ChessPiece {
    private String type;
    private String color;
    private int x;
    private int y;

    // constructor omitted...

    public String getColor() { return color; }
    public int getX() { return x; }
    public int getY() { return y; }

    public boolean move(int newX, int newY) {
        // ...
    }

}
```

Interfaces

An **interface** introduces an **abstraction** for a logical entity.

- An interface consists of a collection of method signatures.
- An interface is **implemented** by one or more classes.
  - Each implementing class must implement the methods specified by the interface.
  - A **class** that implements an interface is a **subtype** of that interface.
- An interface cannot be instantiated by itself.

```
interface ChessPiece {
    String getColor();
    int getX();
    int getY();
    boolean move(int newX, int newY);
}
```

Example

```
class King implements ChessPiece {
    // constructor omitted.
    String getColor() { ... }
    int getX() { ... }
    int getY() { ... }
    boolean move(int newX, int newY) {
        // implement the rules for a King.
    }
}

class Queen implements ChessPiece {
    // constructor omitted.
    String getColor() { ... }
    int getX() { ... }
    int getY() { ... }
    boolean move(int newX, int newY) {
        // implement the rules for a Queen.
    }
}
```

Example

```
public class Main {

    public static void main(String[] args){
        ChessPiece king = new King(“Black”);
        ChessPiece queen = new Queen(“Black”);
        tryMove(king, 1, 1);
        tryMove(queen, 1, 1);
    }

    private static tryMove(ChessPiece piece, int x, int y) {
        if (piece.move(x, y)) {
            System.out.println(“Moved piece successfully!”);
        }
    }

}
```

Example

```
class King implements ChessPiece {
    // constructor omitted.
    String getColor() { ... }
    int getX() { ... }
    int getY() { ... }
    boolean move(int newX, int newY) {
        // implement the rules for a King.
    }
}

class Queen implements ChessPiece {
    // constructor omitted.
    String getColor() { ... }
    int getX() { ... }
    int getY() { ... }
    boolean move(int newX, int newY) {
        // implement the rules for a Queen.
    }
}
```



Duplication!

Class Inheritance

We can also let a class **inherit** from another class:

```
class ChessPiece {
    protected String color;
    protected int x;
    protected int y;

    ChessPiece(String color, int x, int y) {
        // omitted ...
    }

    int getX() { ... }
    int getY() { ... }
    String getColor() { ... }
    boolean move(int newX, int newY) {
      return false;
    }
}
```

Example

```
class King extends ChessPiece {
    ChessPiece(String color, int x, int y) {
        super(color, x, y);
    }

    @Override
    boolean move(int newX, int newY) {
      // implement the rules for a King.
    }
}
```

Abstract Classes

An **abstract class** is a concept between an **interface** and a **class**.

An abstract class can have **fields and methods** (like classes) but it can also have **unimplemented methods** like (like interfaces).

```
abstract class ChessPiece {
    protected String color;
    protected int x;
    protected int y;

    // omitted ...

    abstract boolean move(int newX, int newY);
}
```